Cloud-Native & Scalable Architecture Styles

Mastering Software Architecture Patterns and System Design

Rahul Rajat Singh

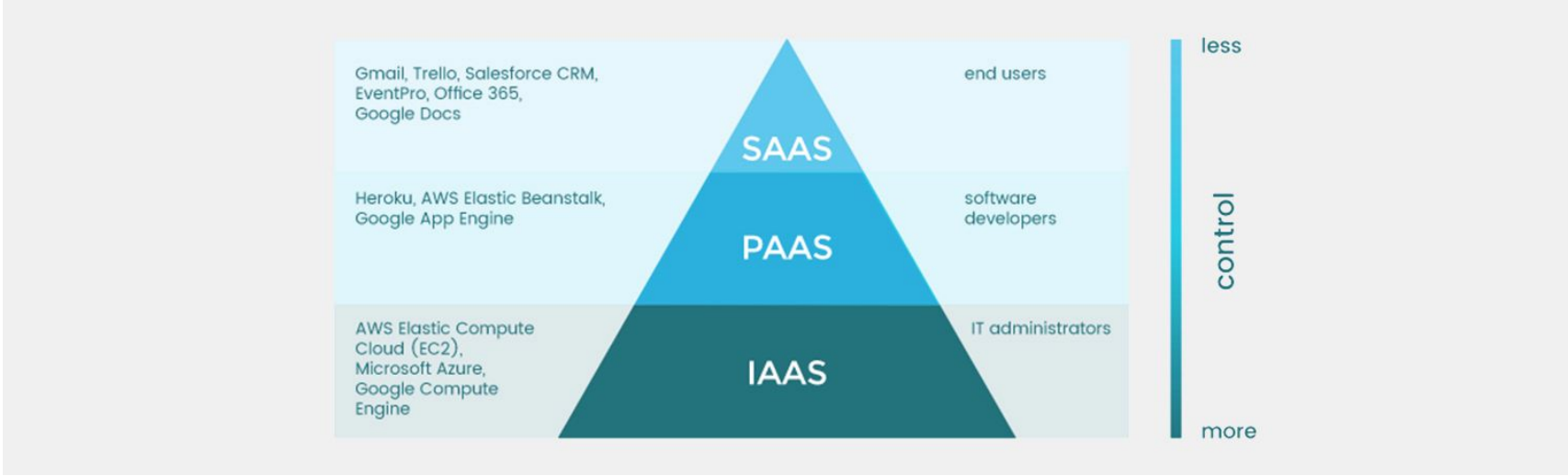
Cloud Service Models – IaaS, PaaS, SaaS, and Beyond

Cloud-Native & Scalable Architecture Styles

Mastering Software Architecture Patterns and System Design

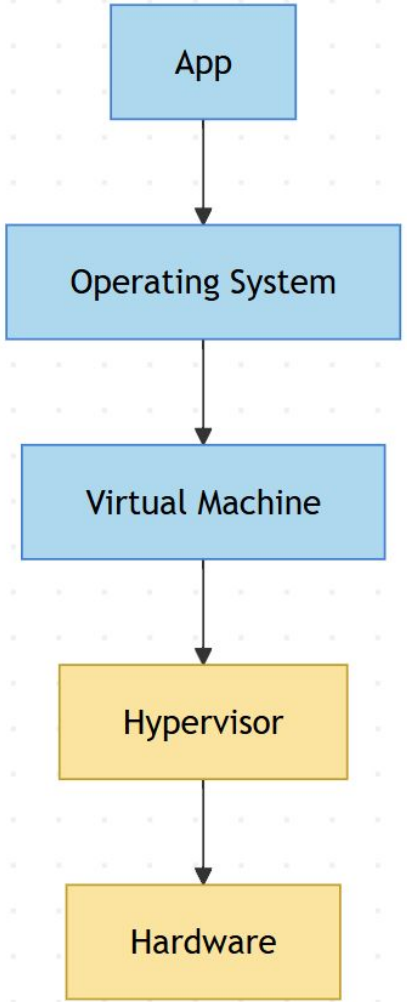
Introduction to Cloud Service Models

- Cloud services deliver computing over the internet.
- Different levels of abstraction: IaaS, PaaS, SaaS.
- Key to modern software delivery and scalability.
- Choosing the right model affects cost, control, and development speed.



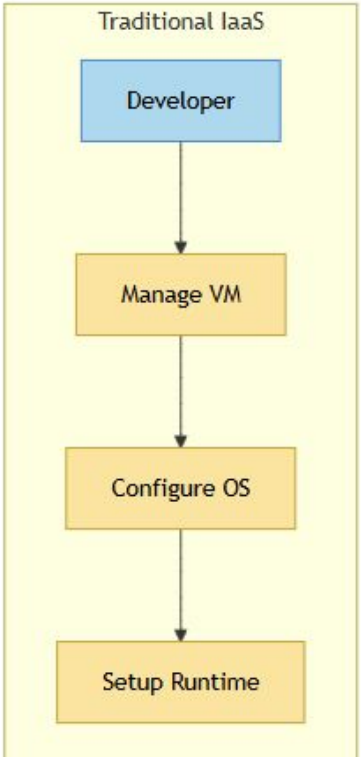
IaaS – Infrastructure as a Service

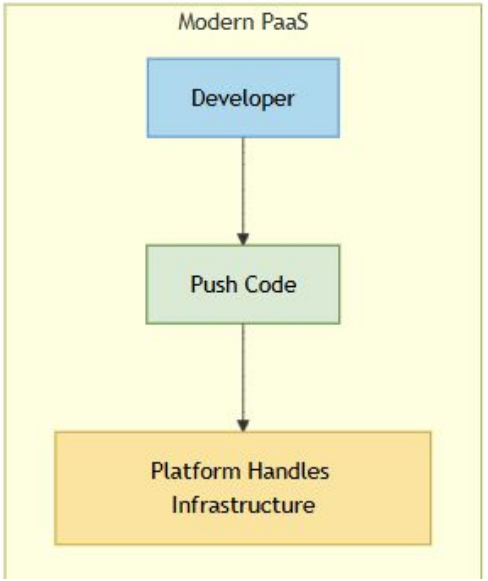
- Raw computing resources: VMs, storage, networks.
- Full control over OS, middleware, runtime.
- Highly flexible; you manage provisioning and scaling.
- Examples: AWS EC2, Google Compute Engine, Azure VM.



PaaS – Platform as a Service

- Abstracts away OS and runtime management.
- Focus on app logic, not infrastructure.
- Automatically handles scaling, patching, provisioning.
- Examples: Heroku, Google App Engine, Azure App Service.





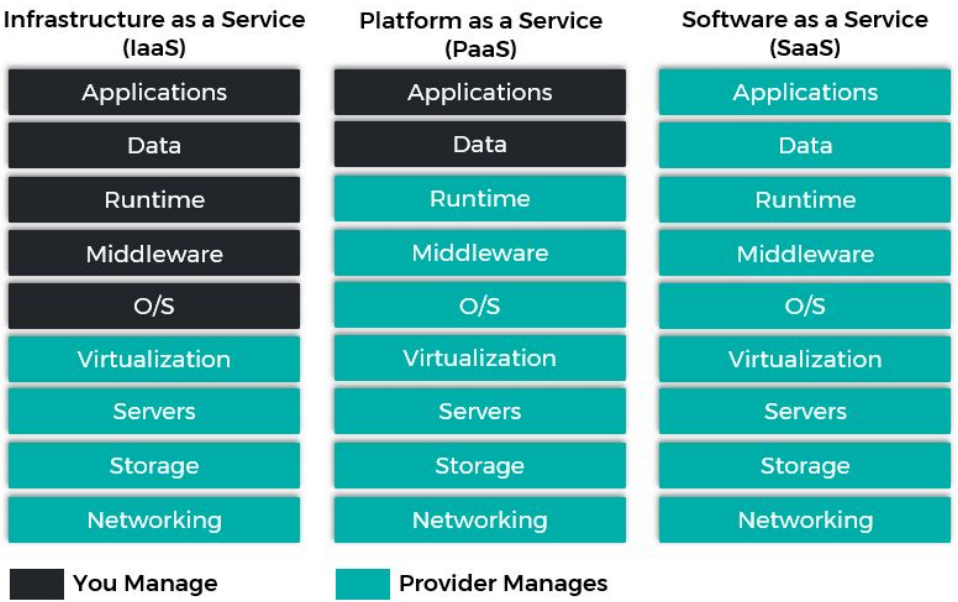
SaaS – Software as a Service

- Fully managed applications — use, don’t build.
- No responsibility for infrastructure or deployment.
- Ideal for standard business functions.
- Examples: Gmail, Dropbox, Salesforce.



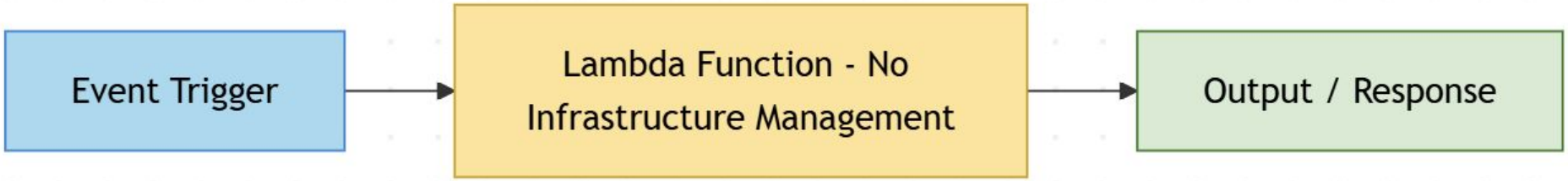
Comparing IaaS, PaaS, and SaaS

- Control vs. convenience trade-off.
- Development speed increases as you go up the stack.
- Security, cost, and scaling handled differently at each level.
- Consider use case, team expertise, and app lifecycle.



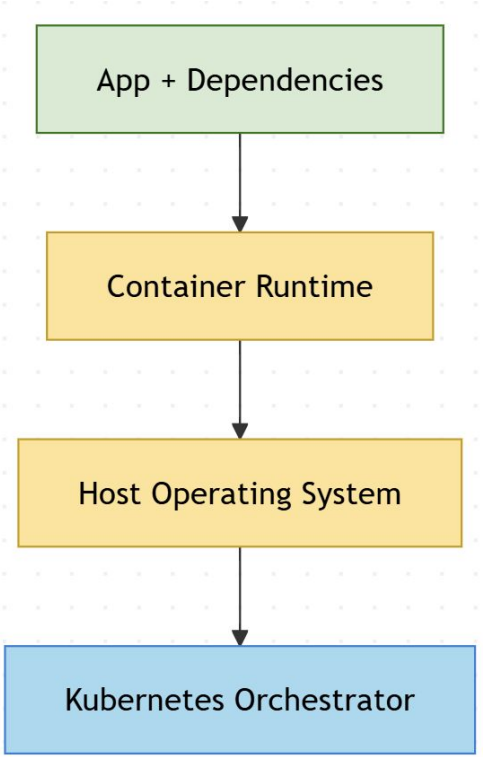
Beyond the Basics – Serverless Computing

- No server management; code runs in response to events.
- Pay only for execution time.
- Ideal for microservices and event-driven workflows.
- Examples: AWS Lambda, Azure Functions, Google Cloud Functions.



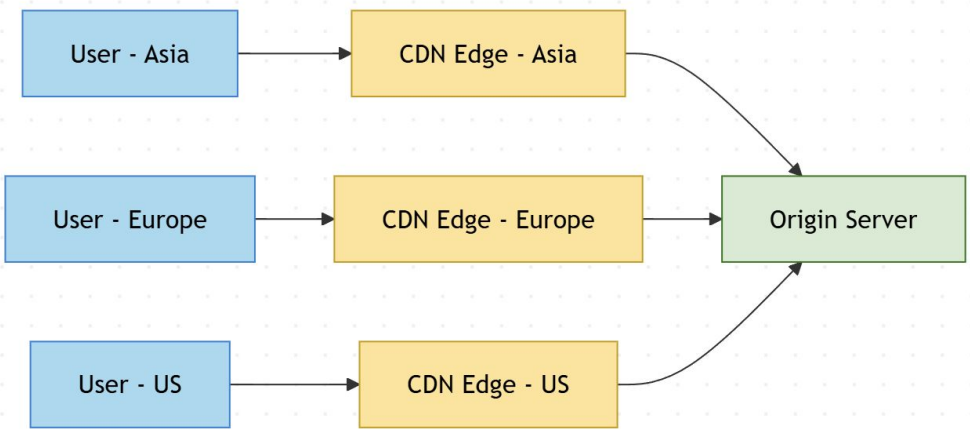
Containers and Cloud Services

- Containers package app + dependencies together.
- More portable than VMs; faster start-up.
- Used across IaaS, PaaS, and FaaS.
- Managed through orchestration tools like Kubernetes.



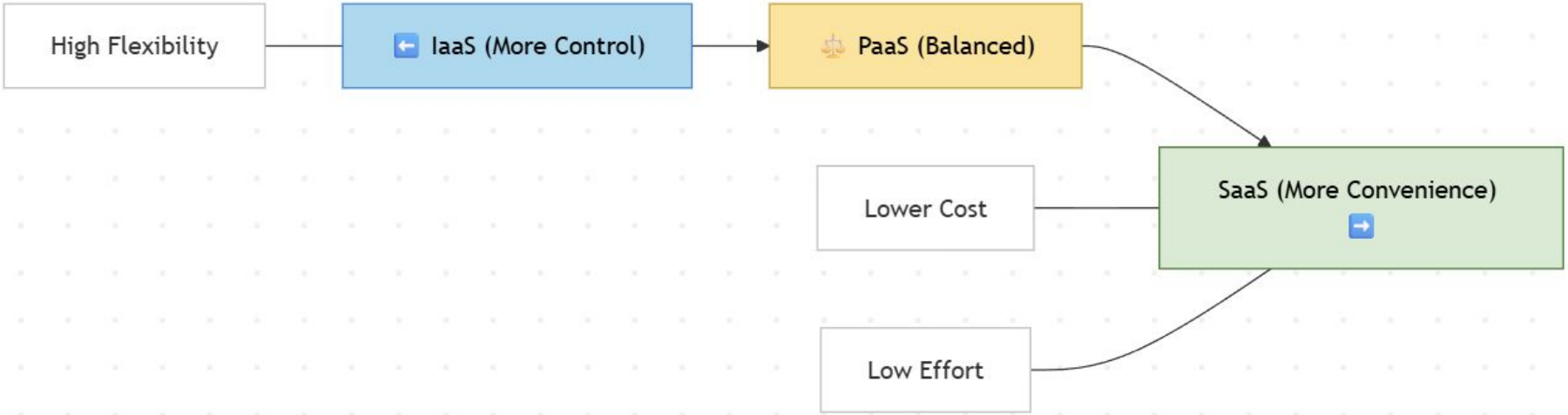
CDNs – Accelerating Cloud Delivery

- Distribute static content closer to users.
- Reduce latency, bandwidth, and server load.
- Works alongside all cloud models.
- Examples: Cloudflare, AWS CloudFront, Akamai.



Trade-offs – Abstraction vs. Control

- Higher abstraction = less control, but faster delivery.
- IaaS: more control, more complexity.
- PaaS/SaaS: less control, more speed and simplicity.
- Consider compliance, customization, and scaling needs.



Summary - Cloud Service Models

- IaaS → best for custom, infrastructure-heavy apps.
- PaaS → ideal for fast development and deployment.
- SaaS → best when you just need functionality.
- Serverless and containers = modern extensions.
- Choose based on team needs, project scale, and constraints.
- What's next:
  - 12-Factor App Design

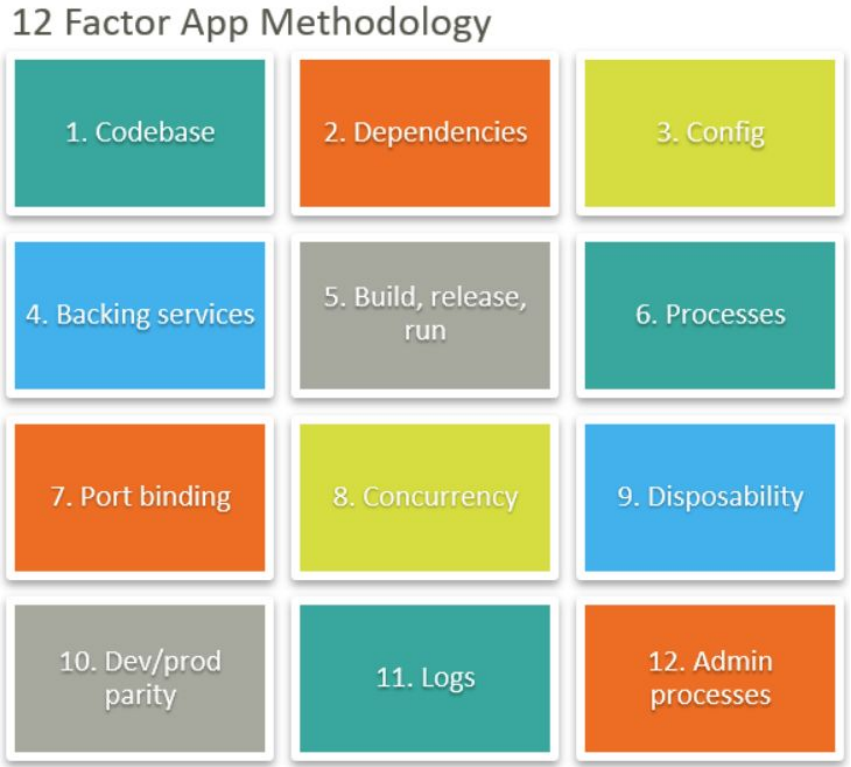
12-Factor App Design

Cloud-Native & Scalable Architecture Styles

Mastering Software Architecture Patterns and System Design

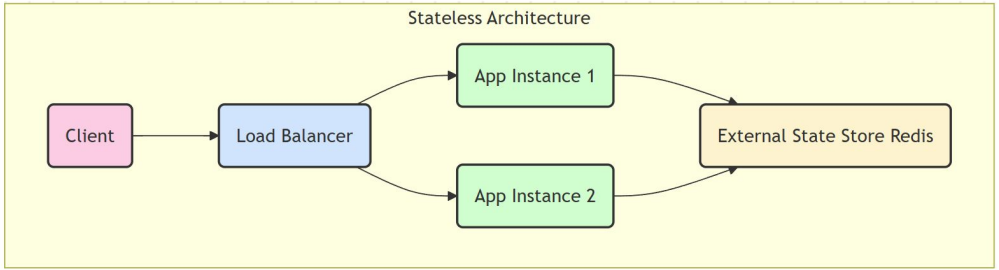
Introduction to 12-Factor App Design

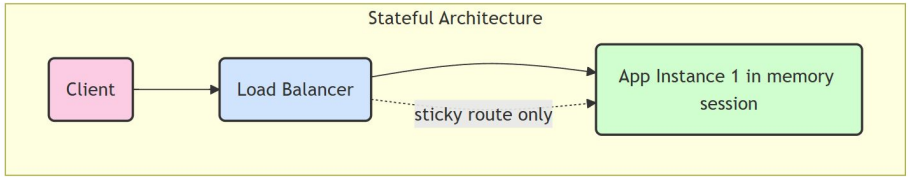
- Modern methodology for building SaaS applications
- Created by Heroku developers
- Designed for scalability, maintainability, and deployability
- Encourages microservice and cloud-native practices



Factor 1 – Stateless Processes

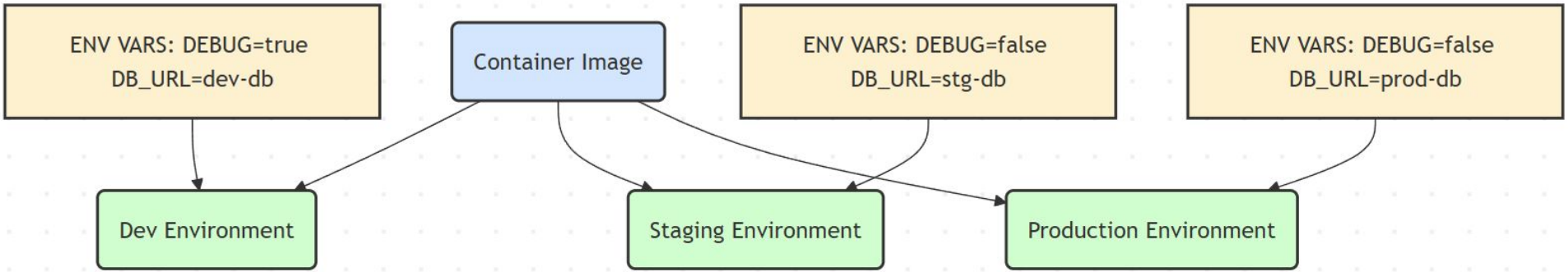
- Application instances should not store local state
- User session/state should be stored in external services (e.g., Redis)
- Enables horizontal scaling
- Resilience through process disposability





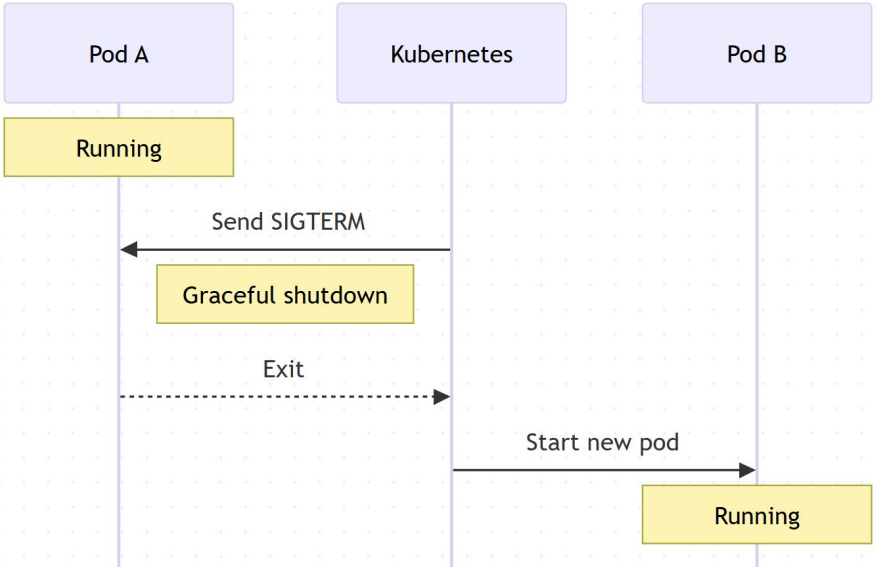
Factor 2 – Externalized Configuration

- Store config in environment variables
- Never hardcode config or secrets
- Enables same build for all environments (dev, staging, prod)
- Supports secure secret rotation



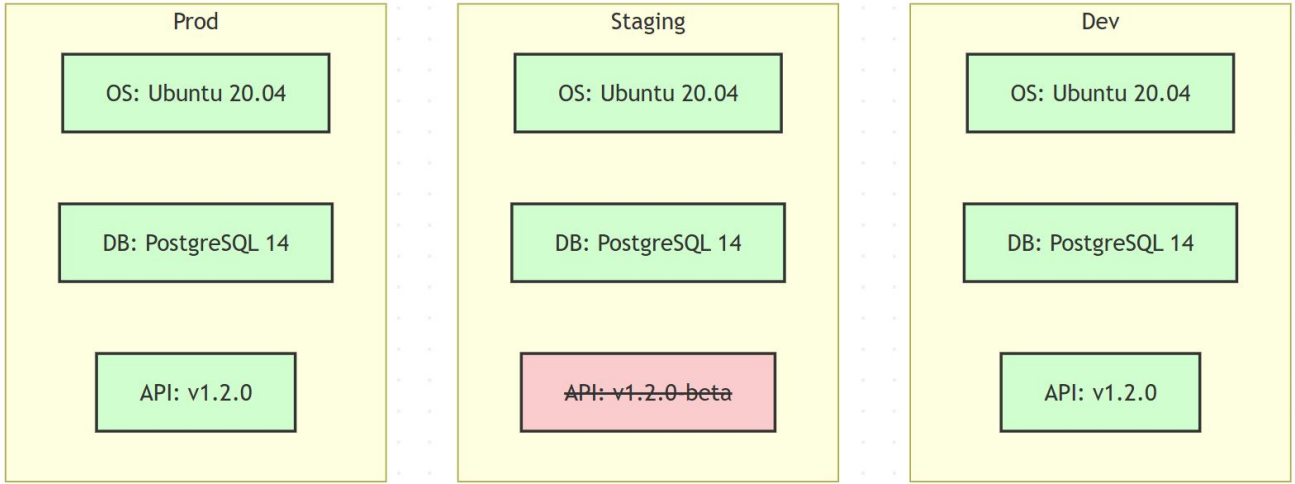
Factor 3 – Disposability

- Fast startup and graceful shutdown
- Helps with rapid scaling and deployments
- Processes should be disposable and restartable
- Encourages fault tolerance



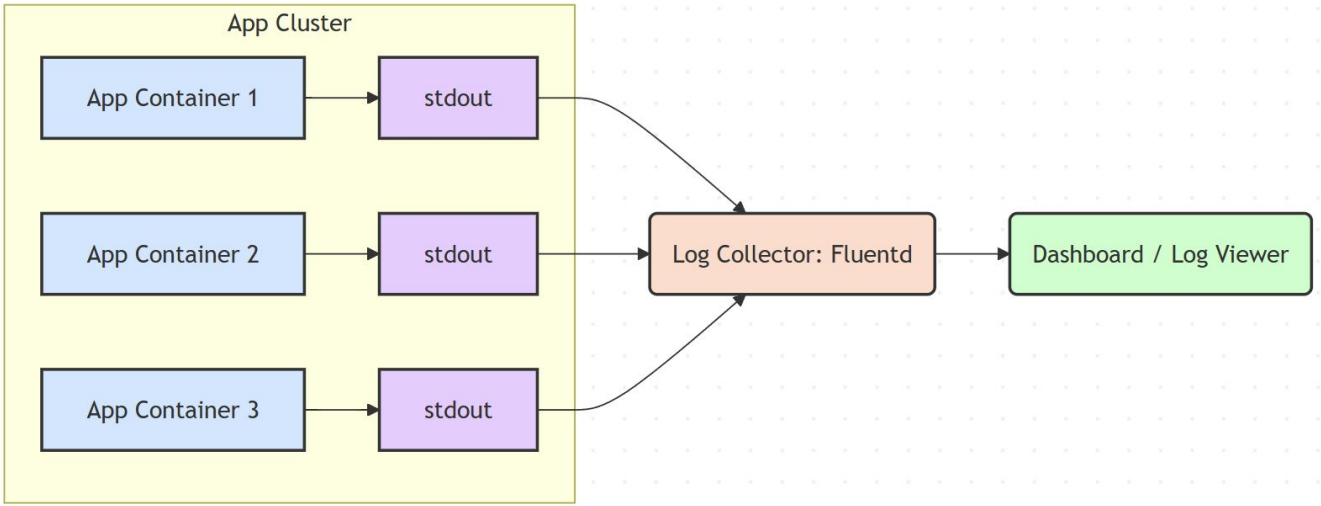
Factor 4 – Dev/Prod Parity

- Keep development, staging, and production as similar as possible
- Reduces environment-specific bugs
- Avoids “it works on my machine” syndrome
- Frequent code deployment keeps parity tight



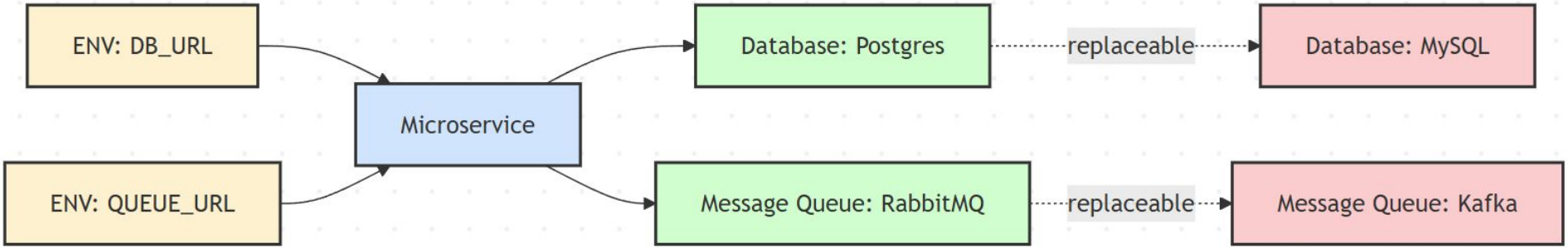
Factor 5 – Logging as Event Streams

- Treat logs as event streams, not files
- Do not manage log files inside the app
- Stream logs to external systems (e.g., ELK, Datadog)
- Centralized logging improves observability



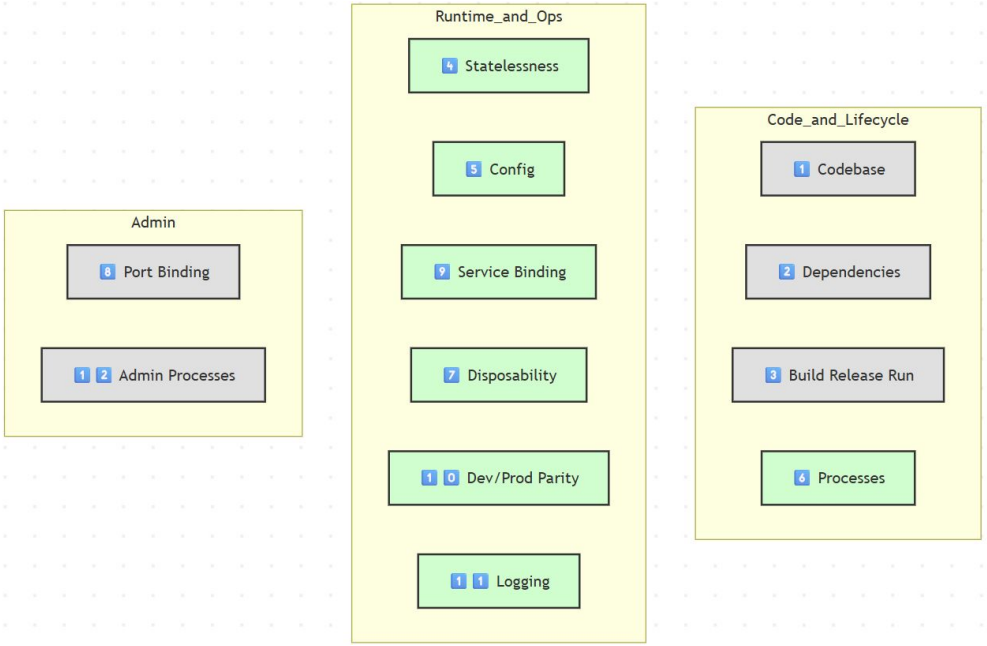
Factor 6 – Service Binding

- Treat backing services (DBs, caches, MQs) as attached resources
- Bind services using environment variables
- Makes swapping services seamless
- Promotes modular, decoupled architecture



The Other 6 Factors (Quick Overview)

- **Codebase** – One codebase tracked in version control, many deploys
- **Dependencies** – Explicitly declare and isolate dependencies
- **Build, Release, Run** – Strictly separate build, release, and run stages
- **Processes** – Execute the app as one or more stateless processes
- **Port Binding** – Export services via port binding (e.g., HTTP server inside the app)
- **Admin Processes** – Run admin/management tasks as one-off processes



Summary – 12-Factor App Design

- 12-Factor App is a modern blueprint for scalable SaaS apps
- Today’s focus: 6 runtime/infrastructure factors
- Remaining 6 support structure, build, and admin tasks
- Adopt incrementally — even partial adoption adds value
- Foundation for microservices, containers, and cloud-native systems
- What’s next:
  - Serverless Architecture Explained

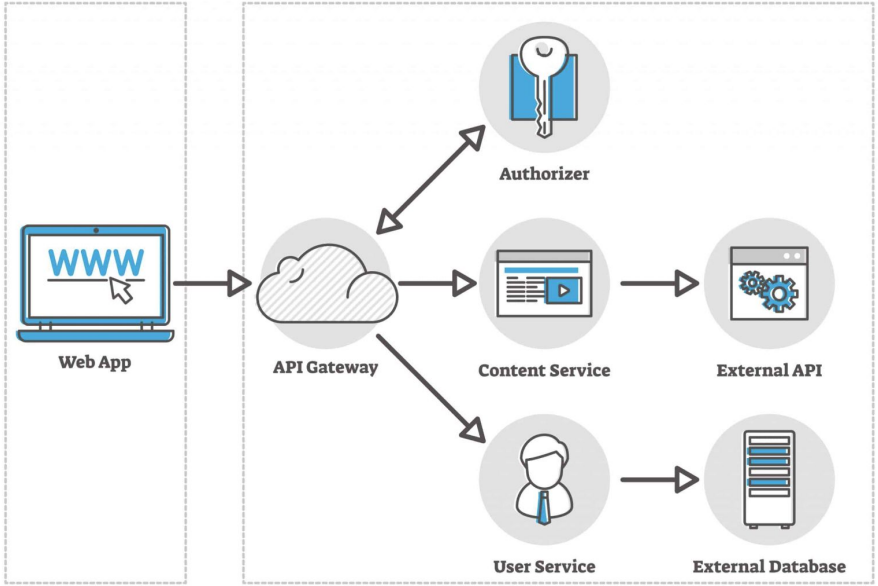
Serverless Architecture Explained

Cloud-Native & Scalable Architecture Styles

Mastering Software Architecture Patterns and System Design

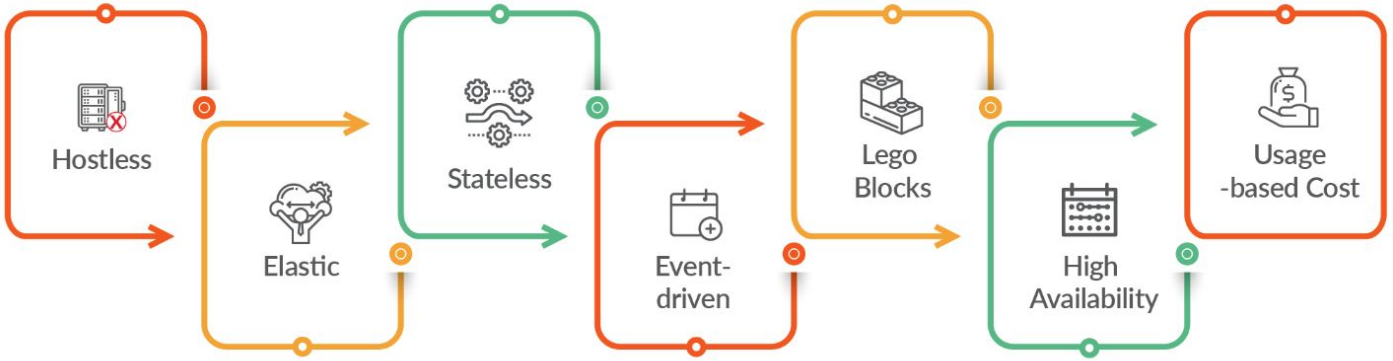
Why “Serverless”?

- Serverless ≠ no servers – ops abstracted away
- Pay per-execution rather than provisioned capacity
- Auto-scales to zero and to peak in seconds
- Tight fit with event-driven application styles



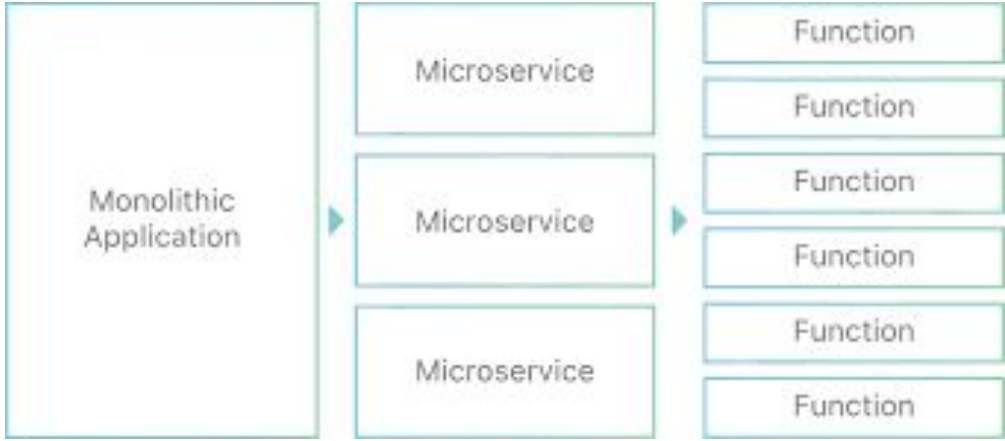
Core Traits of Serverless Platforms

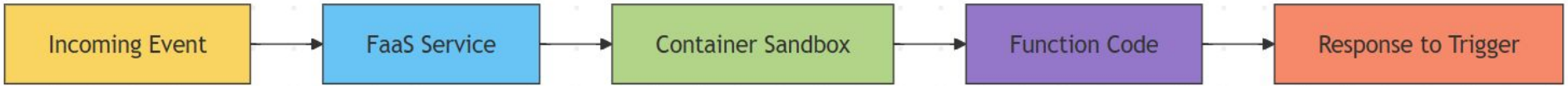
- Managed runtime: infra, patching, scaling handled by provider
- Ephemeral functions: short-lived stateless invocations
- Stateless by design: externalize state (DB, cache, queue)
- Granular billing: billed per 100 ms or request
- Built-in elasticity: scales from 0 → N concurrently



Function-as-a-Service (FaaS) Deep Dive

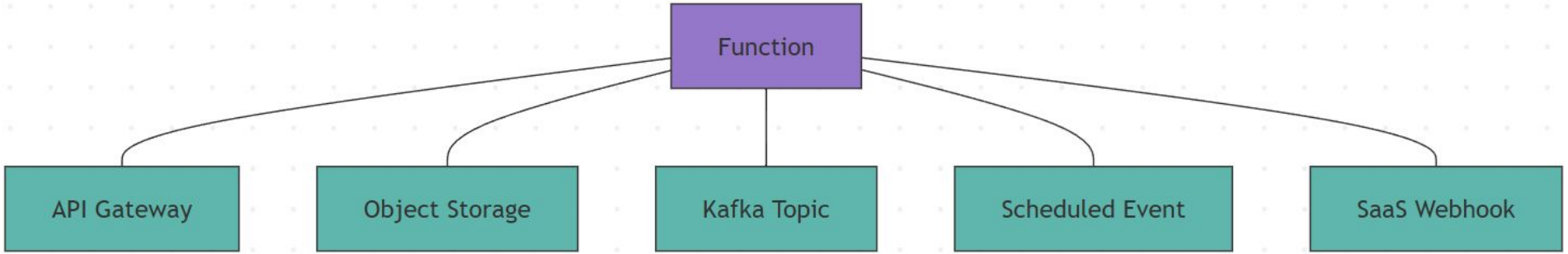
- Small single-purpose functions triggered by events
- Runtime sandbox (AWS Lambda, Azure Functions, GCP Cloud Functions)
- Code package + config (memory, timeout, env vars)
- Concurrency model: 1 event → 1 container instance
- Language/runtime support & limitations (cold start impact)





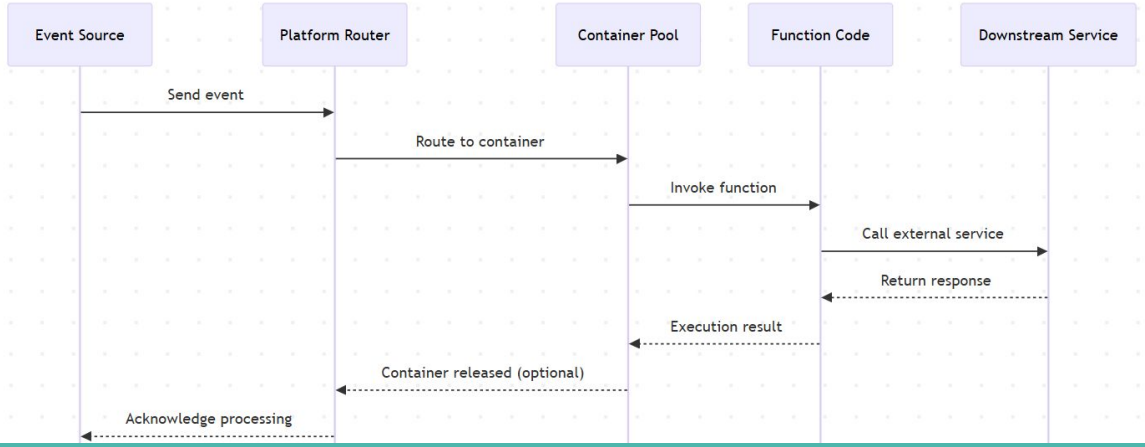
Event Triggers: The Fuel for Functions

- Data sources: HTTP APIs, message queues, object storage, DB streams
- System events: cron/ timers, IoT device telemetry
- Third-party webhooks: Stripe, GitHub, Twilio, etc.
- Trigger mapping handled declaratively in provider console/IaC
- Back-pressure managed by the platform’s queueing/batching



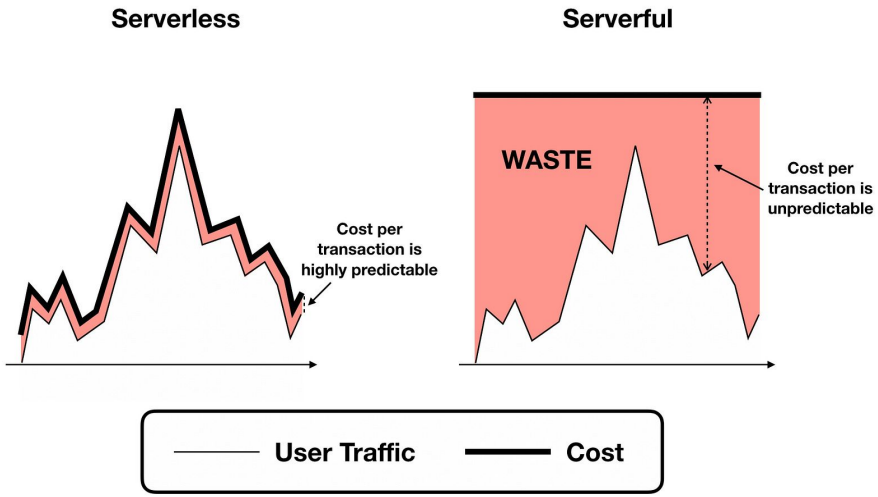
Execution Flow: Life of a Serverless Request

- Event arrives at edge / regional service
- Warm container reused or new container spun up (cold start)
- Function executes with ephemeral storage & context object
- Async vs. Sync invocation paths (fail-retry semantics)
- Auto-emitted logs & metrics to provider observability stack



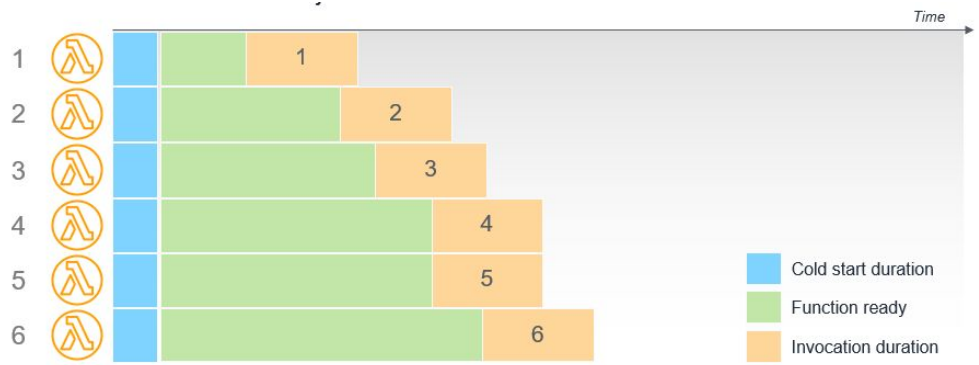
The Serverless Cost Model

- Compute charged per invocation × duration × memory
- No cost when idle — scale-to-zero = $0
- Additional costs: API Gateway, data transfer, managed queue/topic
- Burst traffic can be cheap or surprising — watch request counts
- Effective levers: memory tuning, code optimization, batch/async design



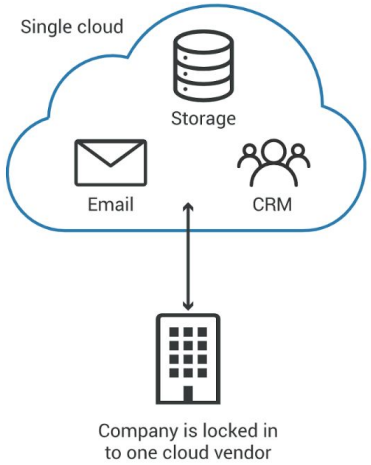
Performance & Cold Starts

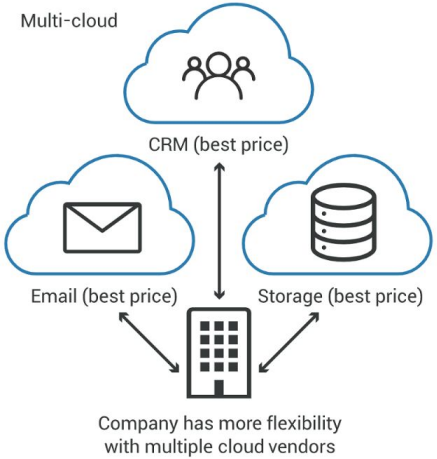
- Cold start latency: language/runtime-dependent (JS < Go < Java)
- Impact of VPC networking & heavy dependencies
- Mitigations: provisioned concurrency, keeping functions small, ARM runtimes
- Warm-start hit ratio crucial for UX; monitor P50/P95
- Real-time vs. batch workloads — decide tolerance thresholds



Vendor Lock-In: Myth or Inevitable?

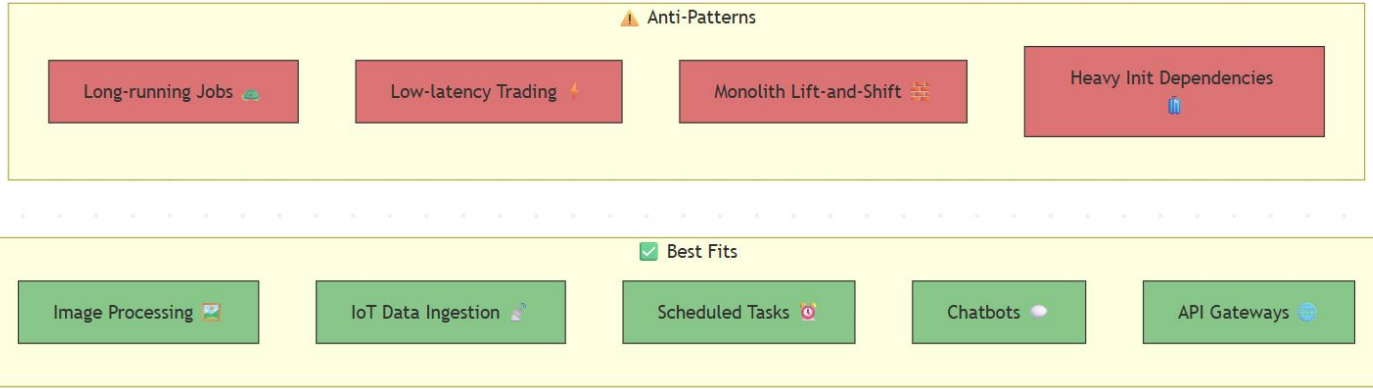
- Proprietary event models & IAM policies tie you to provider
- Cloud-native services (DynamoDB, SQS, Step Functions) deepen coupling
- Portable abstractions: Knative, OpenFaaS, CloudEvents spec
- IaC tooling (Terraform, Pulumi, SST) eases multi-cloud migration
- Acceptable lock-in ↔ business value trade-off (speed vs. flexibility)





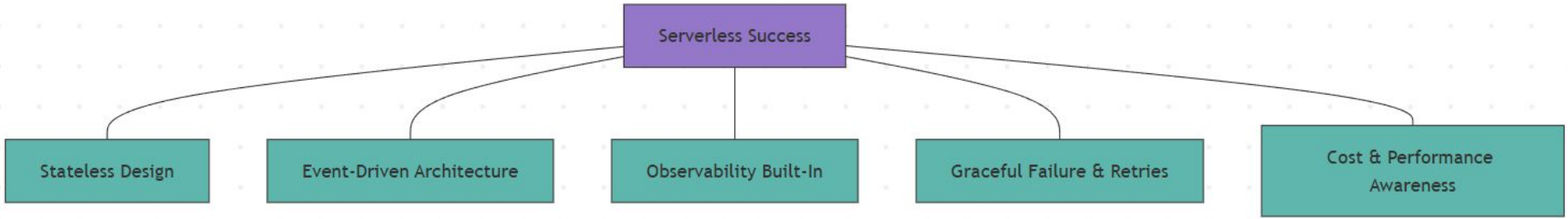
Use Cases & Anti-Patterns

- Good fits: image/video processing, IoT ingestion, scheduled ETL, chatbots
- Edge-friendly: global APIs with API Gateway + Lambda@Edge/Workers
- Anti-patterns: long-running jobs, high-throughput low-latency trading, monolith lift-and-shift
- Integration glue between SaaS products — no servers needed
- Success stories: Netflix thumbnail generator, FINRA market surveillance



Design Principles & Takeaways

- Design for statelessness; externalize session/state
- Event choreography over tight coupling
- Observe everything: structured logs, traces, custom metrics
- Fail gracefully: retries, DLQs, idempotency keys
- Start small, iterate, and measure cost vs. performance continuously



Summary - Serverless Architecture Explained

- Serverless = managed infra + event-driven FaaS
- Understand triggers, flow, cost, and lock-in before adoption
- Use cases abound, but mind cold starts & anti-patterns
- Explore hands-on: AWS Lambda free tier, Google Cloud Run, Azure Functions
- What's next:
  - Containerization with Docker & Kubernetes

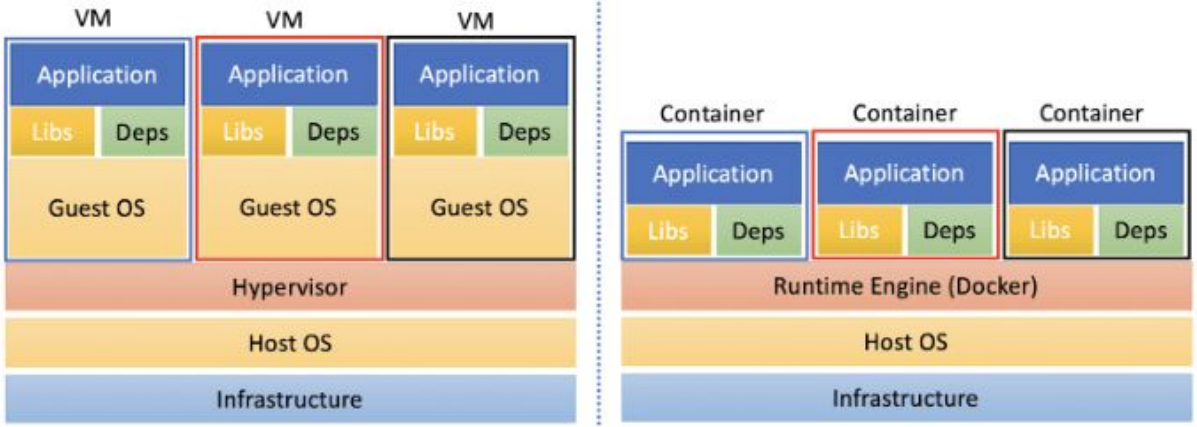
Containerization with Docker & Kubernetes

Cloud-Native & Scalable Architecture Styles

Mastering Software Architecture Patterns and System Design

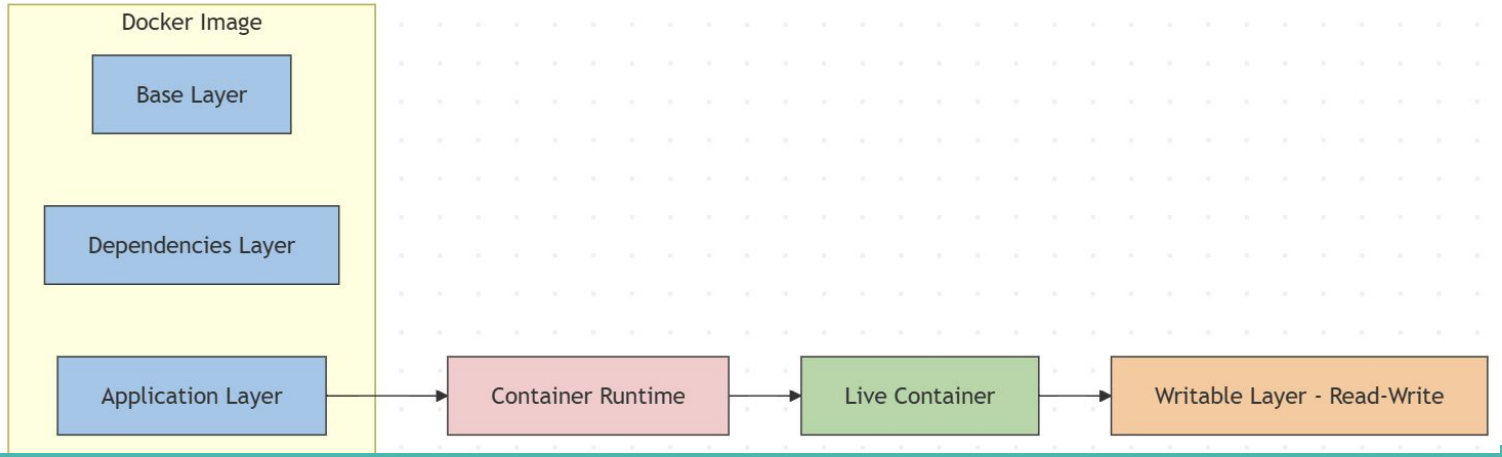
The Case for Containerization

- Traditional deployments: fragile & inconsistent
- “It works on my machine” syndrome
- Scaling apps = config, OS, env hell
- Containers: isolate runtime + dependencies
- Consistency from laptop to production



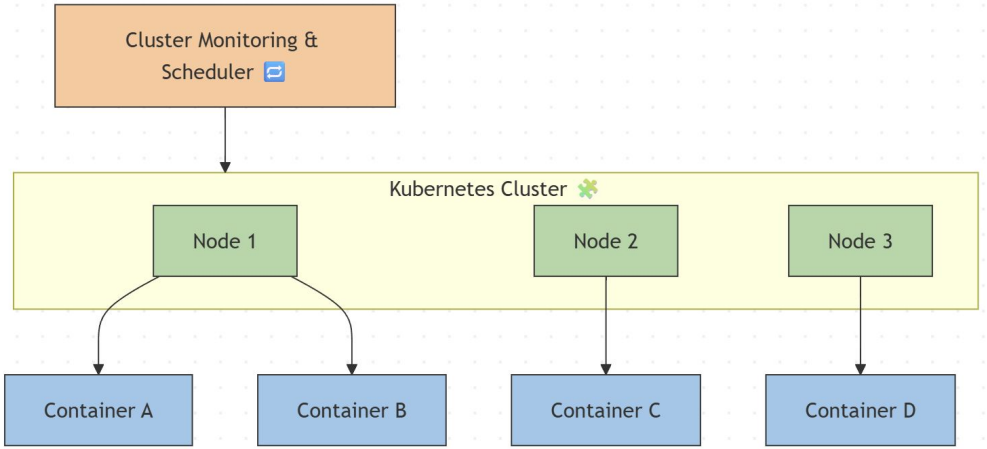
Docker Basics: Images & Containers

- Docker Image = app blueprint (static)
- Container = running instance (dynamic)
- Lightweight, fast startup vs VMs
- Dockerfile defines app environment
- Foundation for CI/CD automation



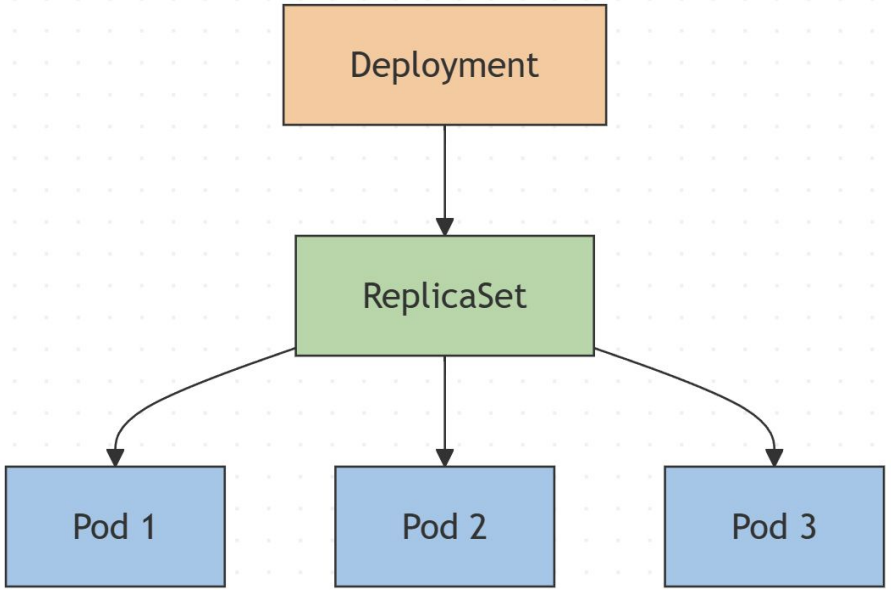
Why Kubernetes?

- One container ≠ production system
- Need for scheduling, scaling, healing
- Kubernetes = orchestrator for container clusters
- Declarative, self-healing, extensible
- Open-source, cloud-agnostic foundation



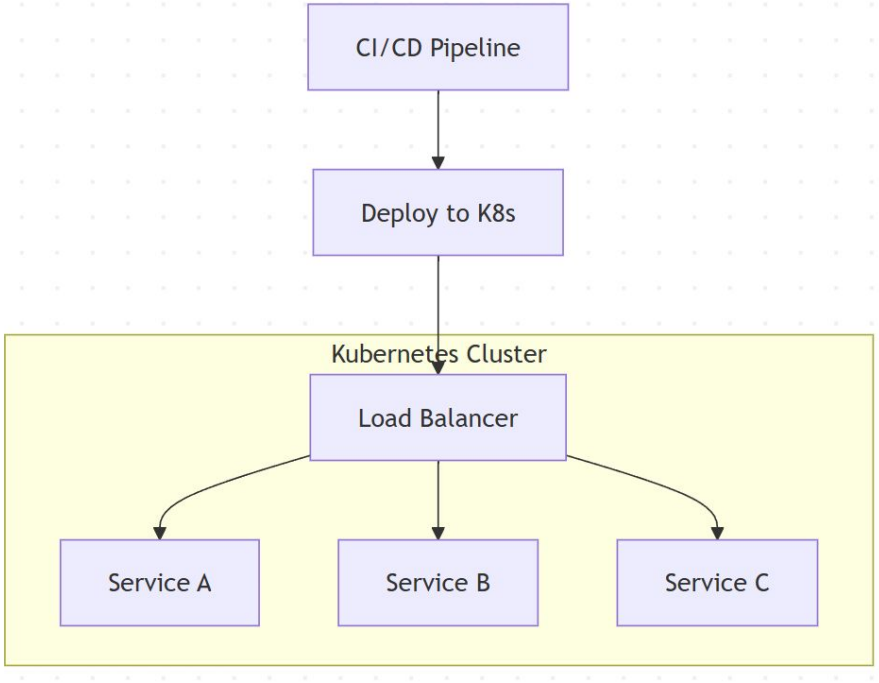
What Kubernetes Gives You Out of the Box

- Auto-scaling & load balancing
- Rolling updates & rollbacks
- Service discovery & networking
- Resource limits & isolation
- Self-healing (restart failed containers)



Why This Matters for System Architecture

- Enables microservices architecture
- Easy horizontal scaling with demand
- Fault isolation through container boundaries
- CI/CD-friendly → fast, repeatable releases
- Cloud-native + hybrid-ready systems



Summary – Containers in Modern Architecture

- Docker = consistent app packaging
- Kubernetes = automated orchestration
- Together: scalable, resilient, repeatable systems
- Key enabler for microservices, cloud-native design
- Architecture = simplified, standardized, production-grade
- What's next:
  - Load Balancing & Autoscaling

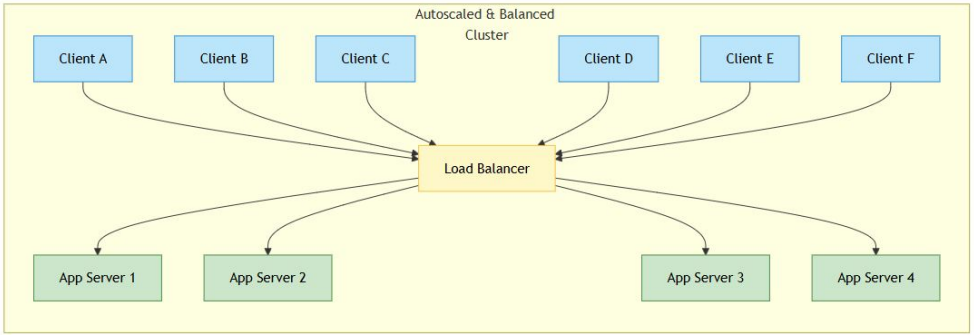
Load Balancing & Autoscaling

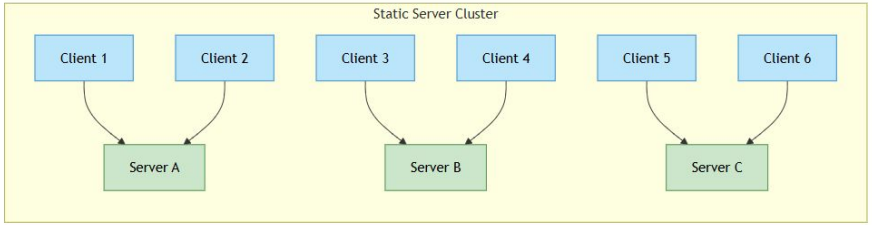
Cloud-Native & Scalable Architecture Styles

Mastering Software Architecture Patterns and System Design

Why Load Balancing & Autoscaling Matter

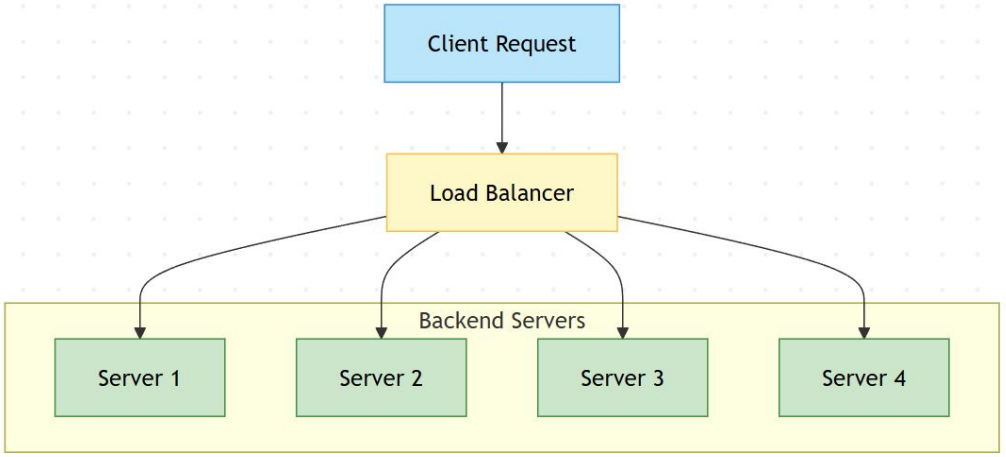
- Modern apps face variable traffic and unpredictable loads
- Manual scaling leads to inefficiency or downtime
- Autoscaling ensures performance + cost-efficiency
- Load balancers distribute traffic to prevent overload
- Essential for cloud-native, resilient systems





What is Load Balancing?

- Distributes incoming requests across multiple servers
- Increases reliability, availability, and throughput
- Prevents any single server from becoming a bottleneck
- Works at different layers: L4 (Transport), L7 (Application)
- Can be hardware-based or software/cloud-native

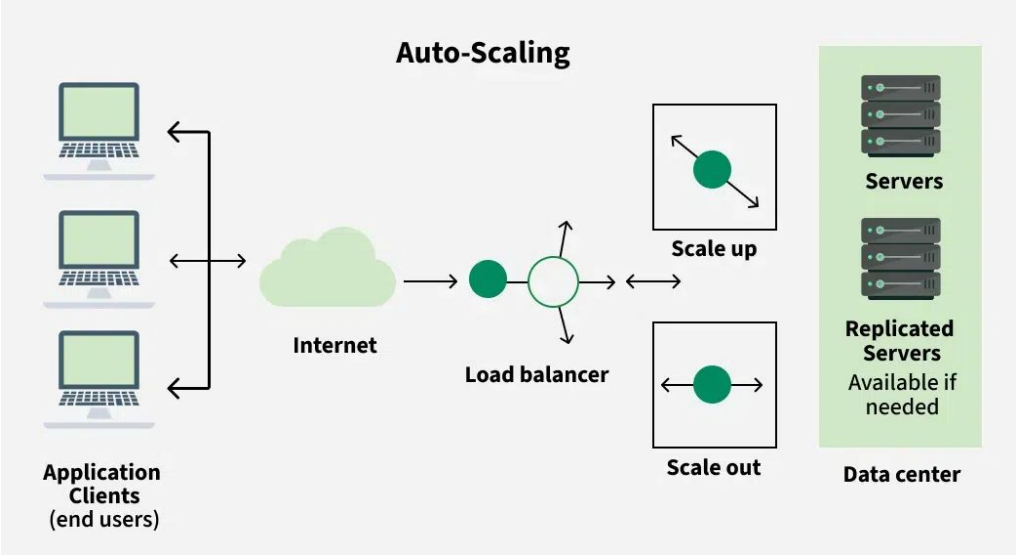


Load Balancing Algorithms

- Round Robin — Equal request distribution
- Least Connections — Chooses server with least active sessions
- IP Hashing — Ensures session stickiness
- Weighted — Based on server capacity/priority
- Random — Simple, less predictable

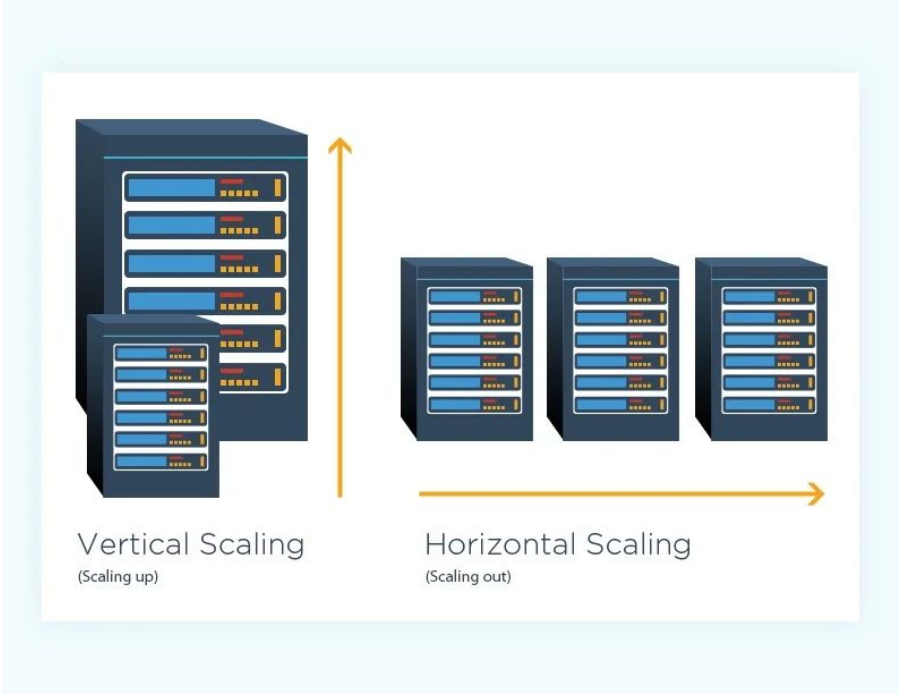
What is Autoscaling?

- Dynamically adjusts the number of instances based on load
- Reduces costs by scaling down during idle periods
- Helps meet SLAs by scaling up during spikes
- Works with metrics: CPU, memory, queue length, etc.
- Core part of elasticity in cloud-native systems



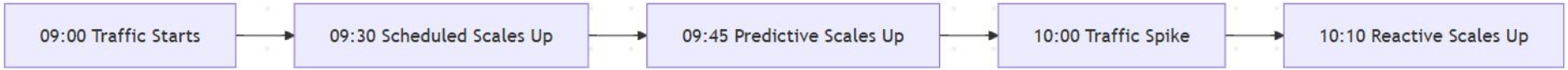
Horizontal vs. Vertical Scaling

- Horizontal (Scale Out): Add more instances
- Vertical (Scale Up): Increase resources on existing node
- Horizontal preferred for cloud-native systems
- Vertical hits limits quickly and needs downtime
- Autoscaling typically = horizontal



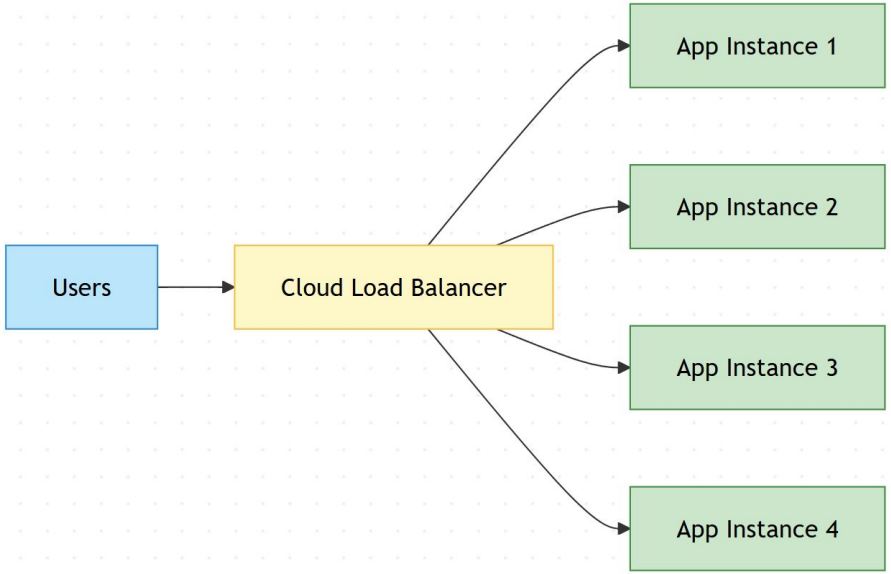
Autoscaling Patterns in the Cloud

- Reactive Scaling: Based on real-time metrics
- Predictive Scaling: Uses historical patterns (ML/forecasting)
- Scheduled Scaling: Fixed known demand periods
- Container Autoscaling: Pod-level or Fargate-style
- Function Autoscaling: Per-invocation scaling (e.g., Lambda)



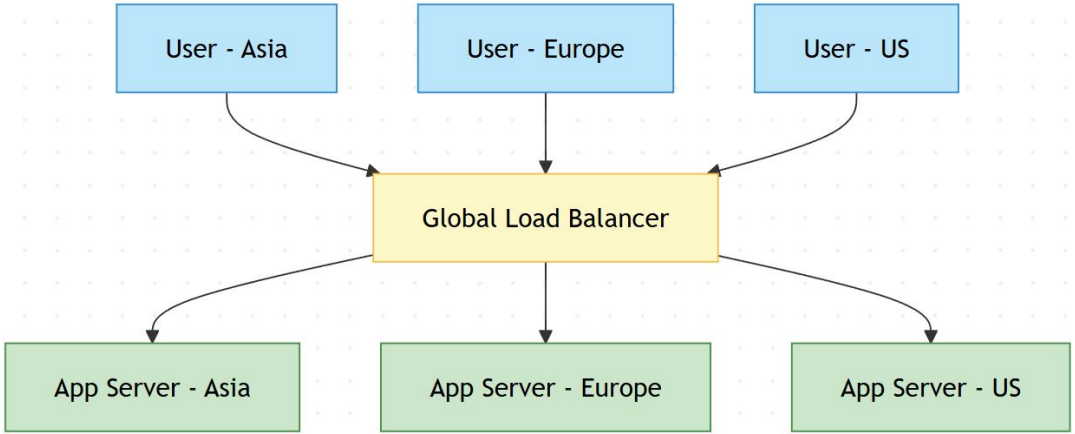
Load Balancers in the Cloud

- AWS: ALB (L7), NLB (L4), CLB (legacy)
- GCP: HTTP(S), SSL proxy, TCP/UDP LBs
- Azure: Front Door, Application Gateway
- Auto-integrated with autoscaling groups
- Can also route based on geo, latency, weights



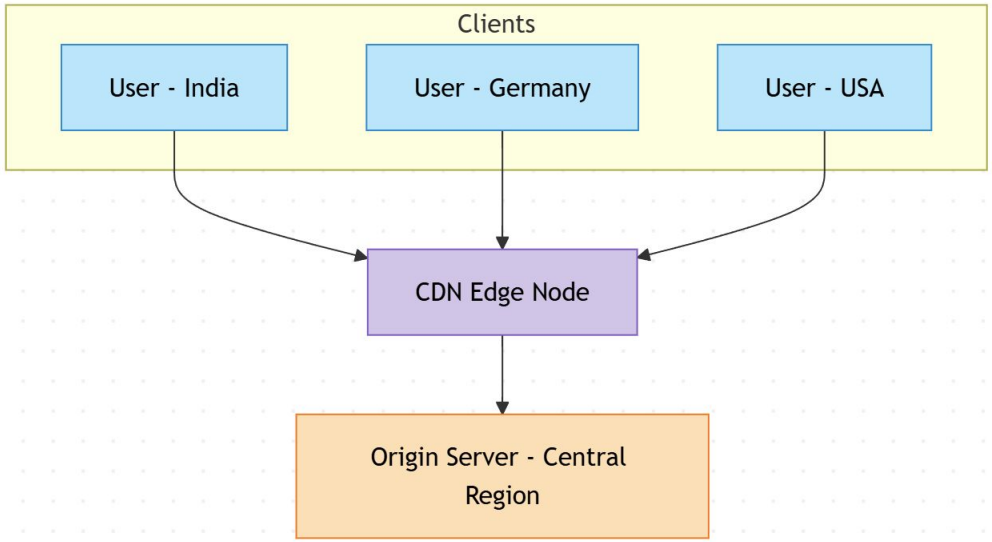
Traffic Routing Strategies

- Canary Deployments: Gradual traffic shift to new version
- Blue-Green: Instant switch between old/new
- Weighted Routing: Split traffic 90/10, 70/30, etc.
- Geo Routing: Based on user region
- Latency-Based Routing: Sends user to lowest-latency server



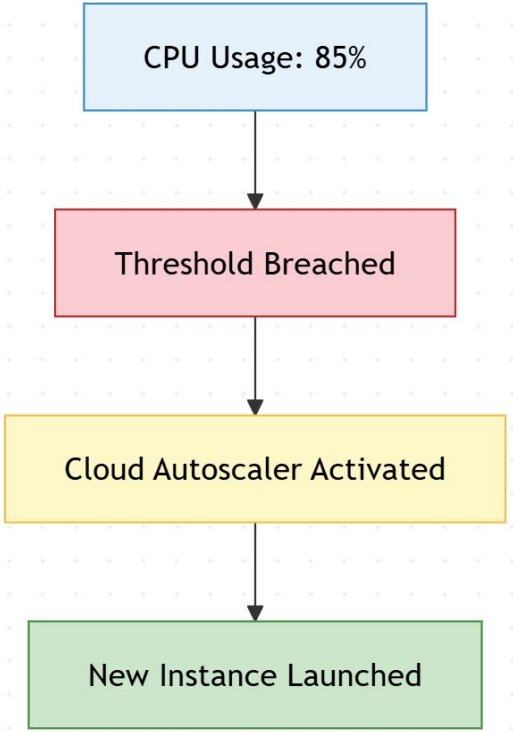
Latency Mitigation Techniques

- Autoscale near user edge (multi-region)
- Use CDNs for static content
- Reduce cold starts in serverless apps
- Connection reuse and keep-alive
- Queue buffering for burst smoothing



Observability in Load Balancing & Scaling

- Track metrics: latency, requests per second, error rates
- Set alerts on thresholds
- Use tools: Prometheus, Grafana, CloudWatch, Datadog
- Visualize autoscaling events and traffic patterns
- Helps debug failures, prevent cascading issues



Summary – Load Balancing & Autoscaling

- Load balancers = smart traffic directors
- Autoscaling = elastic infrastructure on demand
- HPA = Kubernetes-native autoscaling
- Cloud-native patterns combine both for resilience
- Design with latency, cost, and availability in mind
- What's next:
  - Edge Architectures: CDN & Edge Functions

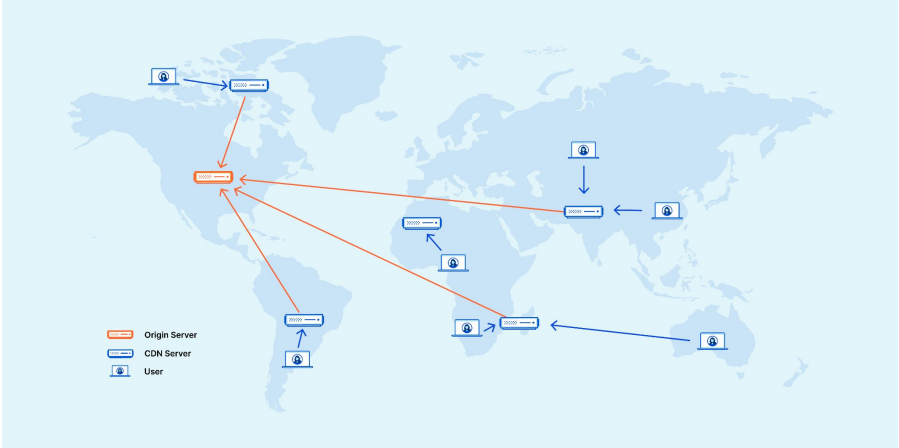
Edge Architectures: CDN & Edge Functions

Cloud-Native & Scalable Architecture Styles

Mastering Software Architecture Patterns and System Design

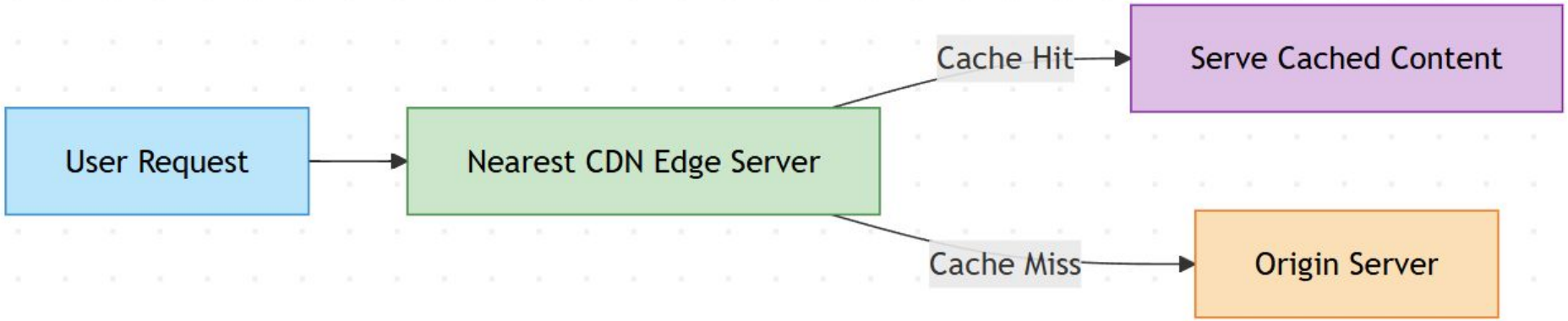
Introduction to Edge Architectures

- Traditional architectures rely heavily on centralized servers.
- Geographical latency affects user experience globally.
- Edge architecture brings compute and data closer to users.
- CDNs and Edge Functions are two key enablers.
- We'll explore both: what they do and why they matter.



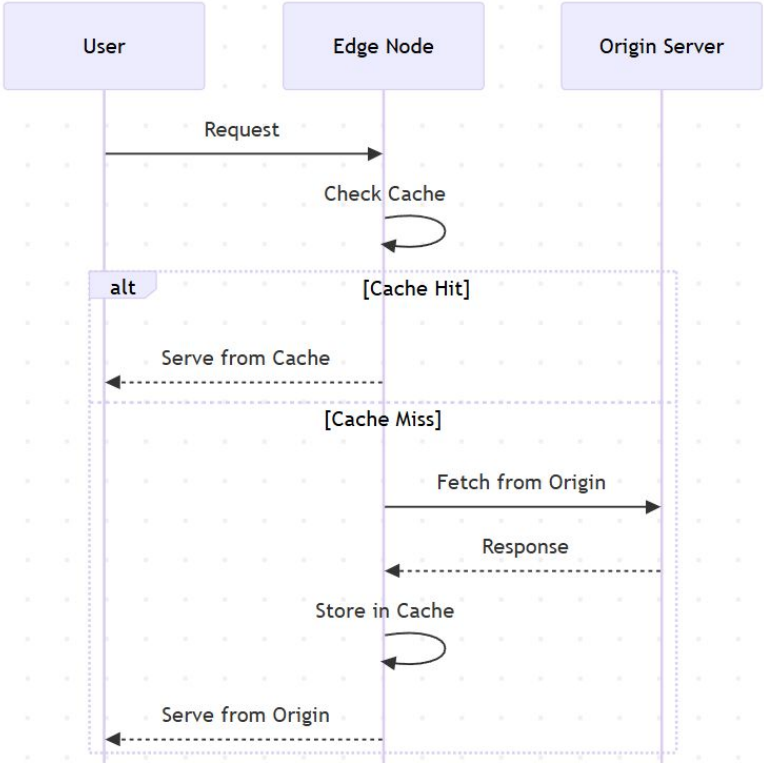
What is a CDN (Content Delivery Network)?

- A globally distributed network of caching servers.
- Delivers static assets (images, JS, CSS, video) from nearest edge.
- Reduces load on origin servers.
- Improves latency, scalability, and reliability.
- Common providers: Cloudflare, Akamai, AWS CloudFront.



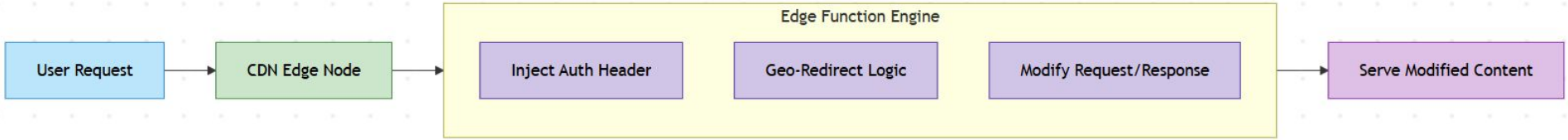
How CDN Caching Works

- Edge nodes cache assets based on rules (e.g., TTL, headers).
- Cache hit = content served directly.
- Cache miss = fetch from origin and store locally.
- CDN purging and cache invalidation manage freshness.
- Edge analytics give cache performance insights.



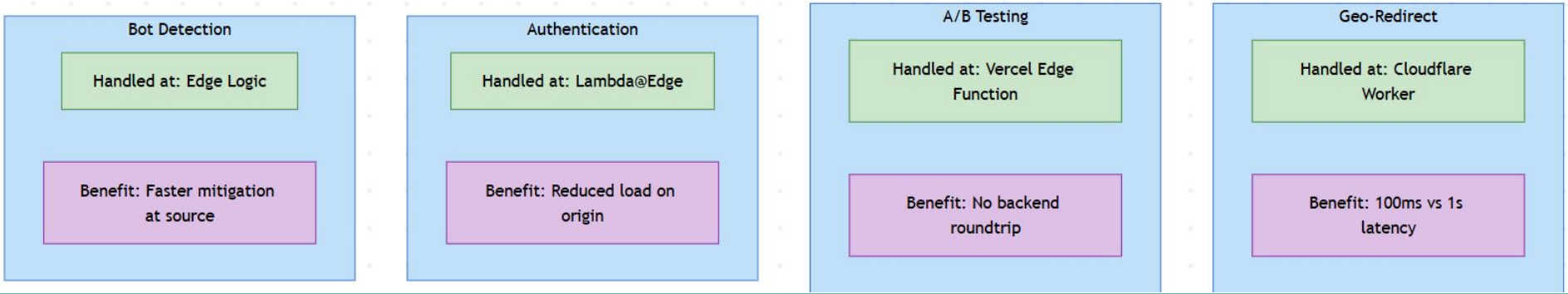
Beyond Static: What Are Edge Functions?

- Lightweight serverless functions running at CDN edges.
- Add dynamic logic close to users (e.g., redirects, auth).
- Lower latency than centralized serverless (e.g., AWS Lambda).
- Stateless and quick—optimized for low cold-start times.
- Examples: Cloudflare Workers, Vercel Edge Functions, Netlify Edge.



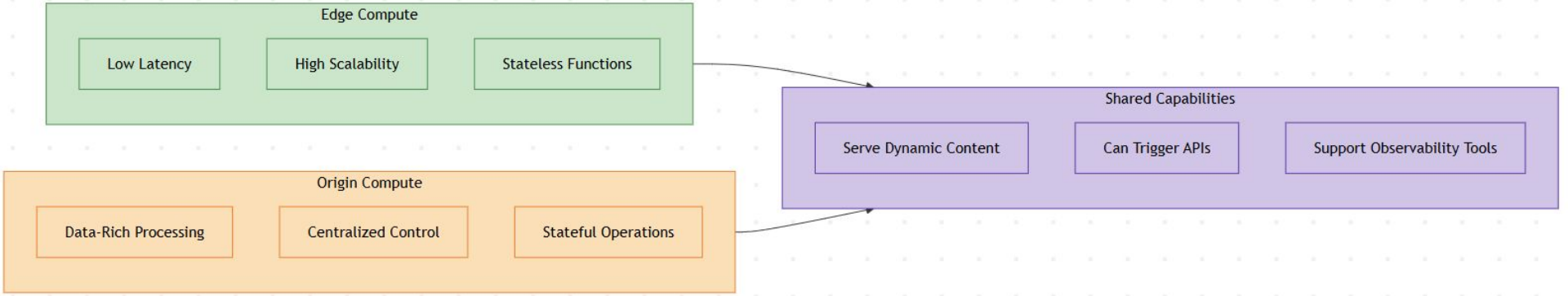
Use Cases of Edge Logic

- Geolocation-based content delivery or language switching.
- A/B testing and personalization without backend calls.
- Edge authentication and token validation.
- Rate limiting and bot detection.
- Real-time analytics preprocessing at the edge.



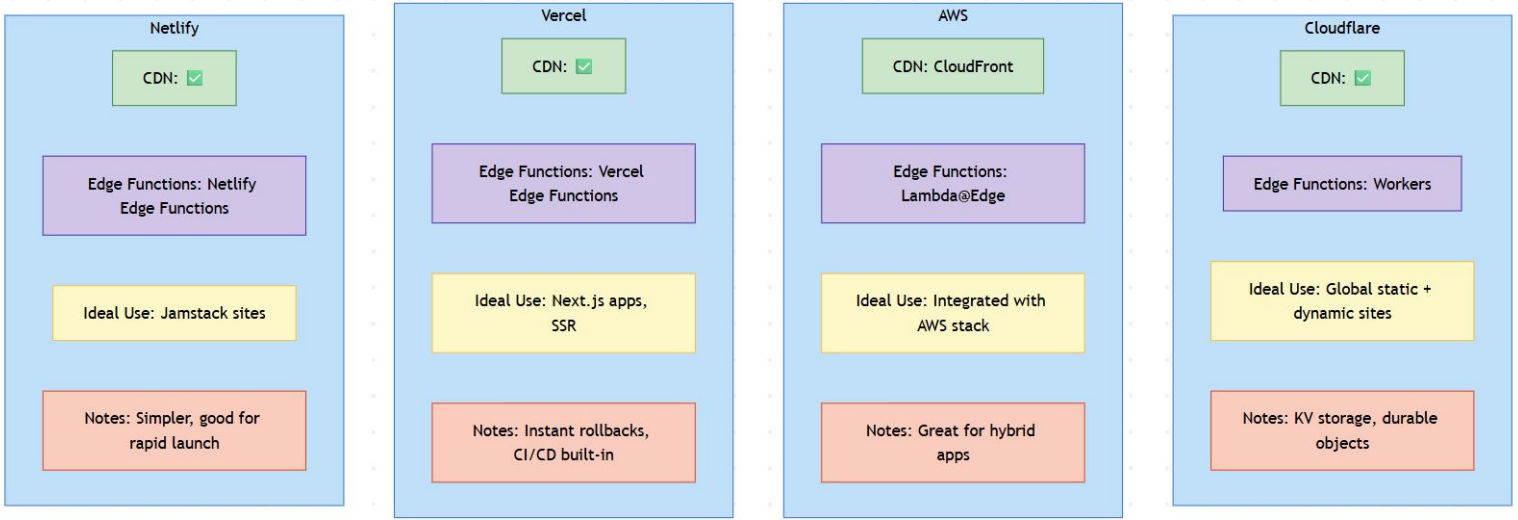
Edge vs. Origin Compute

- Origin compute = centralized (data-rich, powerful).
- Edge compute = distributed (fast, lightweight).
- Trade-offs: speed vs. complexity, consistency vs. proximity.
- Edge is ideal for read-heavy, latency-sensitive tasks.
- Use hybrid models for best results.



CDN and Edge Compute Platforms

- Cloudflare: Global network, Workers, KV storage.
- AWS CloudFront + Lambda@Edge: Tight AWS ecosystem.
- Vercel: Developer-first, React/Next.js optimized.
- Netlify: Jamstack-friendly edge functions.
- Selection depends on ecosystem, performance, and tooling.

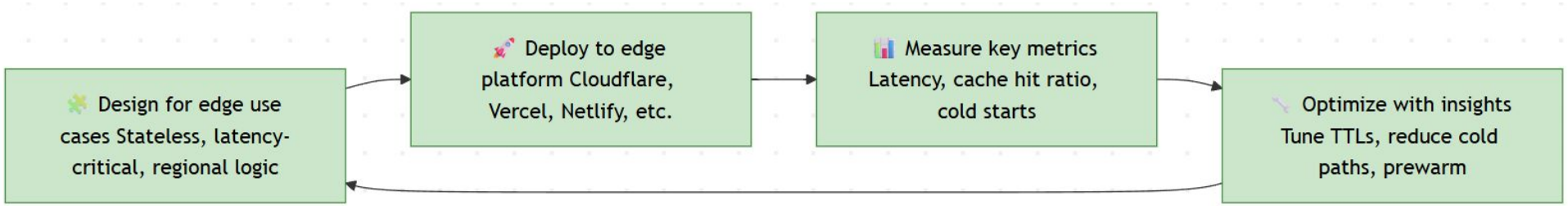


Challenges in Edge Architectures

- Debugging is harder at the edge due to distribution.
- Edge compute has limited runtime (memory, time).
- Cold starts (though reduced) still impact some flows.
- Data consistency and synchronization challenges.
- Monitoring and observability tooling still maturing.

Best Practices for Designing Edge Architectures

- Use edge for latency-critical and stateless logic.
- Minimize external calls from edge functions.
- Leverage CDN caching aggressively before going dynamic.
- Test edge behavior across geographies.
- Monitor cache hit ratio, latency, and function success rates.



Summary - Edge Architectures: CDN & Edge Functions

- CDNs accelerate static delivery through caching.
- Edge functions bring dynamic logic closer to users.
- Together, they reduce latency and increase responsiveness.
- Edge-first design improves global performance at scale.
- What's next:
  - Observability: Logs, Metrics & Tracing

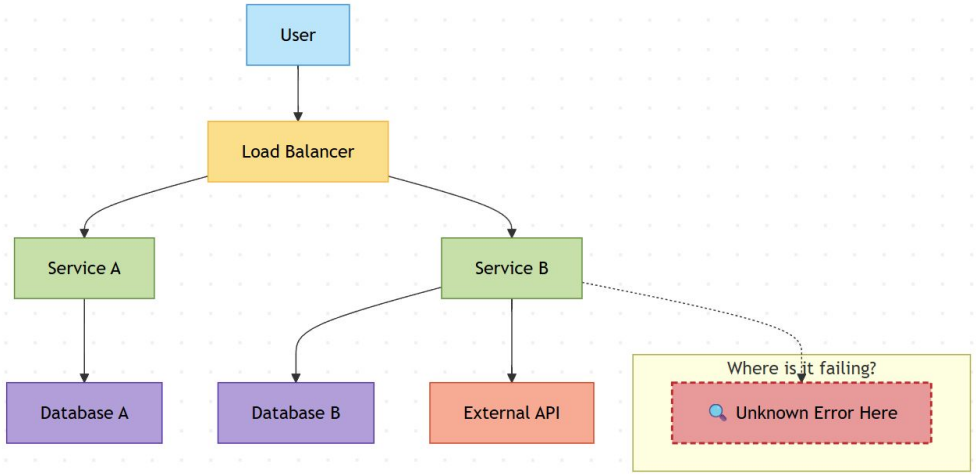
Observability: Logs, Metrics & Tracing

Cloud-Native & Scalable Architecture Styles

Mastering Software Architecture Patterns and System Design

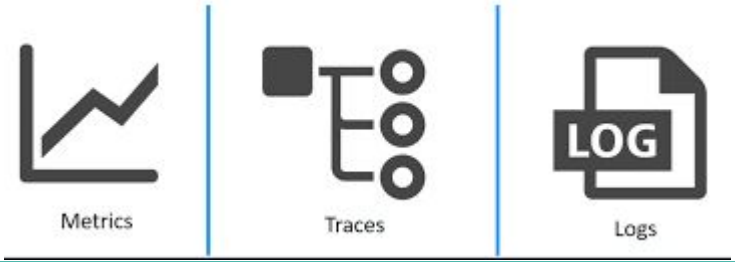
Why Observability Matters in Cloud Architecture

- Complex systems = many moving parts
- Failures happen across services, not in isolation
- Observability = understanding what your system is doing
- Enables faster debugging, better reliability, and smarter scaling
- Core pillars: Logs, Metrics, Tracing



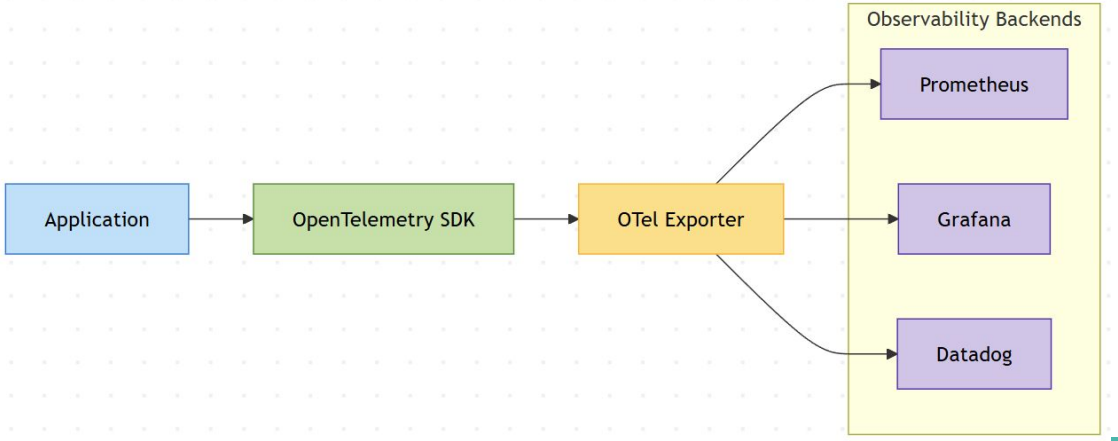
The Pillars of Observability: Logs, Metrics & Tracing

- Logs: Record discrete events and errors
  - Best for root cause analysis
- Metrics: Time-series data (CPU, latency, etc.)
  - Ideal for dashboards and scaling triggers
- Tracing: Follows a request across services
  - Diagnoses latency and service dependencies
- Together, they provide visibility into what, how, and why
- Foundation for alerting, autoscaling, and SLA compliance



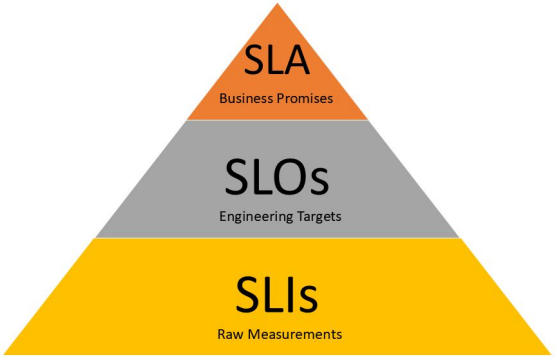
OpenTelemetry – A Unified Observability Framework

- Vendor-neutral, open-source standard
- Supports logs, metrics, traces in one spec
- Helps decouple telemetry from vendor lock-in
- Libraries for all major languages
- Growing adoption across cloud platforms



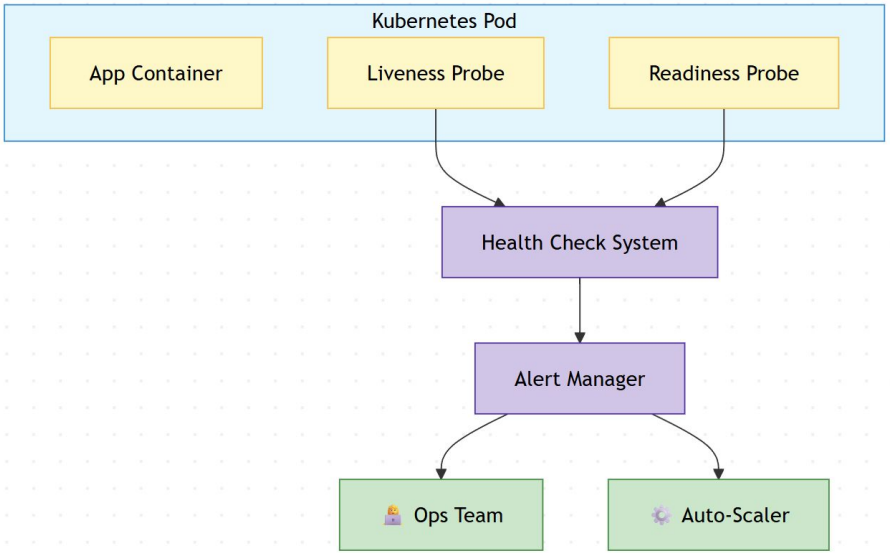
SLIs, SLOs & SLAs – Designing for Reliability

- SLI: measurable indicators (latency, error rate)
- SLO: internal goals (99.9% success in 30d)
- SLA: external commitments (with penalties)
- SLOs drive alerting, scaling, and architecture decisions
- Reliability must be designed, not just monitored



Health Checks & Alerting in Architecture

- Health checks = feedback loop for service status
- Liveness vs. readiness probes (esp. in Kubernetes)
- Alerts tied to SLO thresholds (not just raw metrics)
- Prevent alert fatigue with actionable thresholds
- Drives automated recovery, restarts, and scaling



Summary - Observability for Architects

- Logs = What happened
- Metrics = System health over time
- Tracing = Why a request behaved the way it did
- OpenTelemetry unifies data collection
- Observability drives design, scaling, and recovery
- What’s next:
  - UI Architecture Patterns for Apps